Checkpoint: Comparativo de algoritmos de roteamento de cabos em parques eólicos

Raoni Silva

Sumário

1. Recaptulação do problema
2. Ambiente de testes
3. Comparativo proposto
4. Sweeping
5. Sweeping Multistart
6. Variable Neighbourhood Search
7. Heurística baseada em Sucessive Shortest Path (DIJNOR)

Recaptulação do problema

Especificações:

- O fluxo que sai de uma turbina deve ser suportado por um único cabo.
- São dados diferentes cabos, com diferentes capacidades e custos.
- Cada subestação comporta um número máximo de conexões.
- Cabos não podem se cruzar.

Recaptulação do problema

Objetivo: Conectar todas as turbinas à subestação minimizando o custo total do projeto de cabeamento.

Ambiente de testes

- Ryzen 5 5600, 16Gb de RAM
- Implementação utilizando Rust

Comparativo proposto

Comparar heurísticas baseadas em árvores com heurísticas baseadas em fluxo à custo mínimo

> Sweeping VS SSP adaptado

Comparativo proposto

- Na literatura também existem comparativos de que o VNS é uma das melhores meta-heurísticas para o problema.
- Devido a isso, foi implementado o VNS e testado os dois algoritmos como base para o VNS.

Sweeping

Variáveis: Turbina inicial (t), Tamanho do grupo (`tpg`)

1. Clusterize as turbinas em grupos de tamanho `tpg` por ordem radial;
2. Usando a subestação como nó inicial, adicione ela a cada cluster e rode o algoritmo de prim para encontrar a árvore geradora mínima;
3. Calcule o custo considerando apenas aquele cluster (utilizando uma DFS);
4. Some os custos de cada cluster;

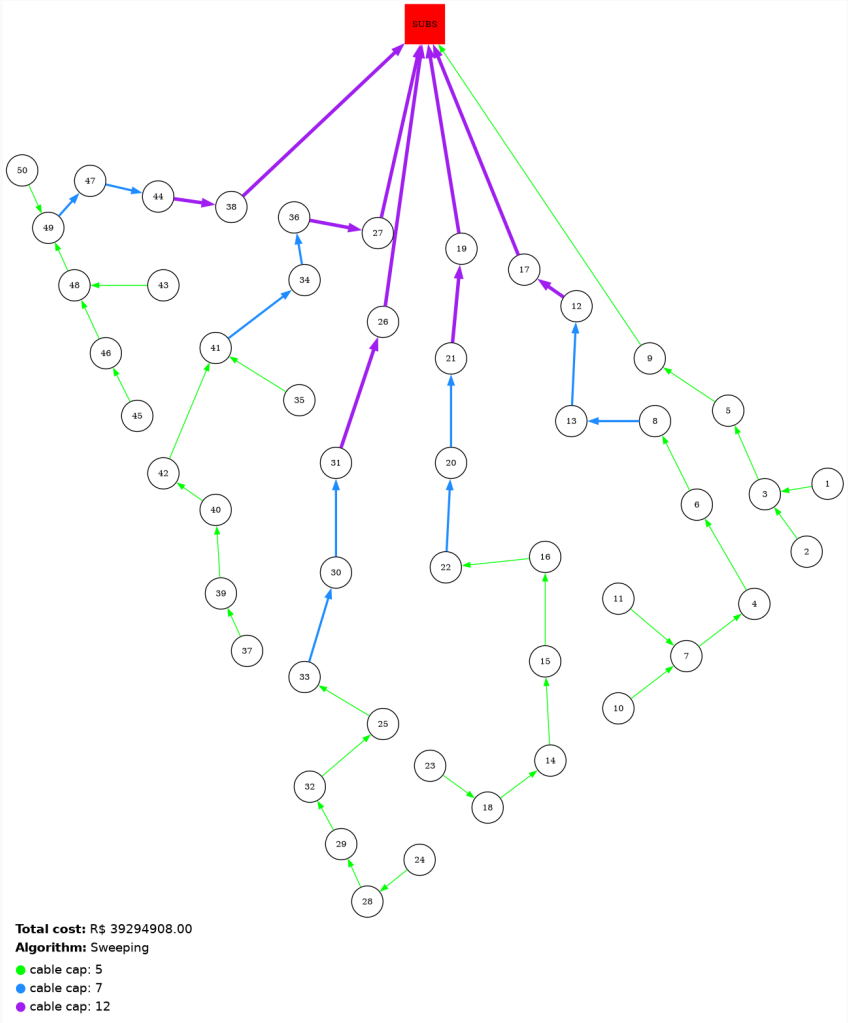
Sweeping

```rust
// Inicializa solução atual
let mut solution: Vec<SolutionEdge> = Vec::new();

// Para cada grupo
for group in groups {
    // Execute o algoritmo de prim inicializado na subestação (*substation)
    let tree = prim_for_group(&group, *substation, graph);
    solution.extend(tree);
}

// Calcule o custo dessa solução.
let cost = calculate_cost(
    &mut solution,
    *substation,
    graph.node_count(),
    max_connections,
    cables,
);
```

Sweeping


SUBS
50
47
44
38
36
27
19
17
12
49
48
43
34
26
21
9
46
41
35
13
8
5
45
31
20
1
42
6
3
40
2
16
22
30
39
4
11
37
15
7
33
10
25
23
14
32
18
29
24
28
Total cost: R$ 39294908.00
Algorithm: Sweeping
cable cap: 5
cable cap: 7
cable cap: 12

Sweeping

> Perceba o agrupamento em ordem ângular dos clusters

Sweeping Multistart

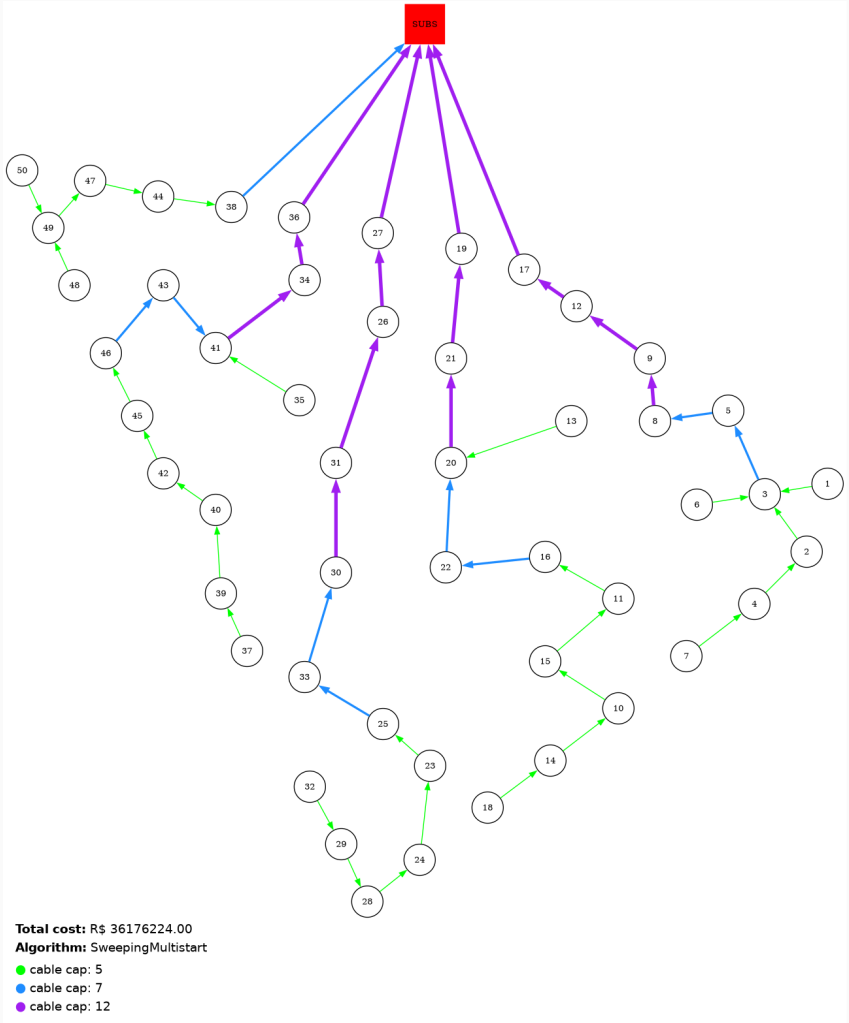
Variáveis: Não contém variáveis

1. Para cada tamanho de grupo possível (`tpg`)
2. Para cada turbina como turbina inicial
3. Para cada direção possível
4. Execute o Sweep
5. Guarde a melhor solução

Sweeping Multistart

```
// Para cada posição inicial.
for start_idx in 0..n_turbines {
    // Para cada direção
    for dir in [1_isize, -1_isize].iter() {
        // Para cada tamanho de grupo possível
        for tpg in start_tpg..=max_tpg {
            // Execute!
            ...
            // Atualize o melhor resultado.
            if cost < best_cost {
                best_cost = cost;
                best_tree = current_tree;
            }
        }
    }
}
```

Sweeping Multistart



Sweeping Multistart

> Perceba que mantém a ordem radial, mas geralmente encontra uma solução melhor

Variable Neighbourhood Search

Variáveis: Uma solução inicial `s`, A quantidade de “shakes” `K`, A quantidade de shakes inicial `Ki`, inicializada em 1.

1. Dê um shake na solução inicial vezes, sem se preocupar se melhora ou piora a solução;
2. Faça uma busca local e guarde a melhor solução para o problema encontrado na vizinhança;
3. Caso tenha melhorado ou `K = 4`, `Ki = 1`, caso contrário, `Ki += 1`;
4. Repita enquanto não tiver estourado o limite de tempo;

Variable Neighbourhood Search

```rust
let start_time = Instant::now();
let mut rng = StdRng::seed_from_u64(self.seed);
let mut incumbent = self.algorithm.execute(problem);

// Solução inicial
let initial_cost = incumbent.cost;

let mut k = 1;

while start_time.elapsed() < self.max_time {
    // 1. Shaking: Agita a solução atual 'k' vezes (aleatório)
    let candidate = self.shake(&mut rng, problem, &incumbent, k);

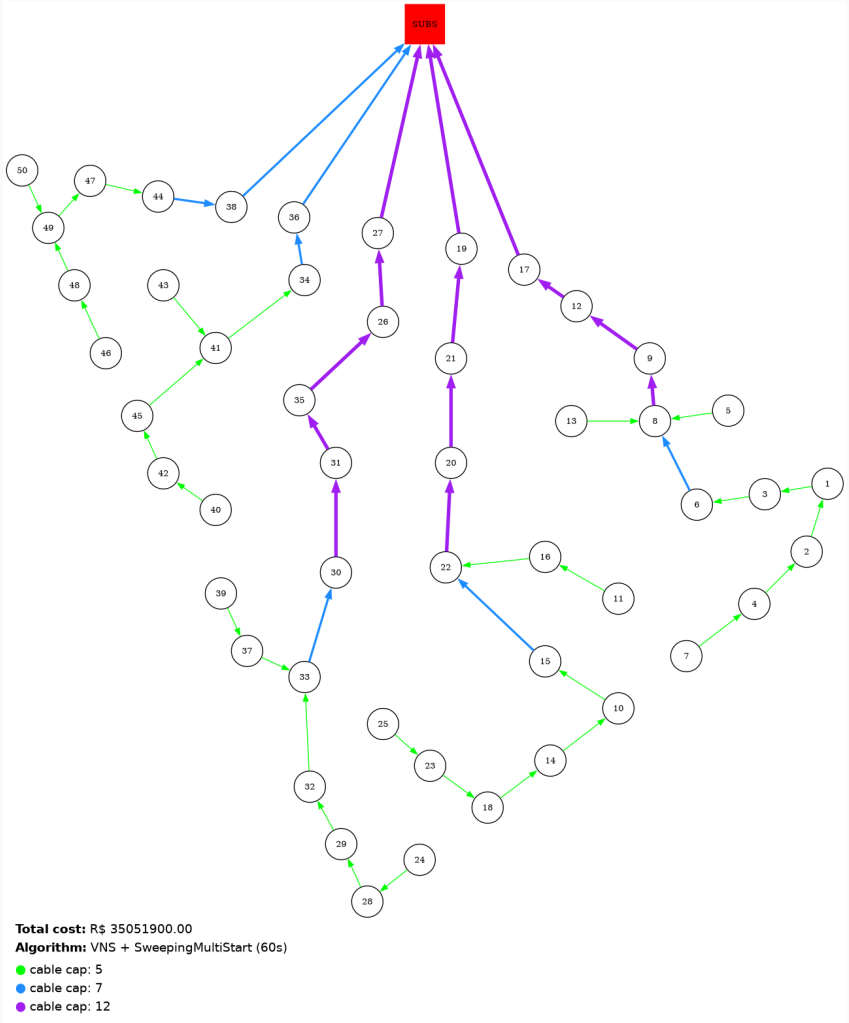
    // 2. Busca Local: Refina a solução agitada até o ótimo local
    let local_optimum = self.local_search(problem, candidate);

    if local_optimum.cost < incumbent.cost {
        incumbent = local_optimum;
        k = 1;
    } else {
        k += 1; // Não melhorou, aumenta a força da agitação
        if k > self.k_max {
            k = 1;
        }
```

Variable Neighbourhood Search

```
    }
}
return incumbent;
```

Variable Neighbourhood Search



Variable Neighbourhood Search

> No pior caso, o VNS é igual a solução inicial, o que significa que não encontrou melhorias

Sucessive Shortest Path (DIJNOR)

Variáveis: Uma ordenação/permutação na ordem de visitação dos nós

1. Para cada turbina, compute o caminho mais curto entre a turbina e a subestação;
2. Aumente uma únidade de fluxo por esse caminho;

Sucessive Shortest Path (DIJNOR)

Como calcular o caminho mais curto?

Sucessive Shortest Path (DIJNOR)

Como calcular o caminho mais curto?

DIKSTRA!

Sucessive Shortest Path (DIJNOR)

Como evitar loops infinitos em ciclos negativos?

Sucessive Shortest Path (DIJNOR)

Como evitar loops infinitos em ciclos negativos?

Sacrifique a otimalidade permitindo que cada SP visite cada nó apenas uma vez.

Sucessive Shortest Path (DIJNOR)

Principais dificuldades na implementação

- Computar corretamente fluxos reversos
- Computar na hora certa o custo considerando cruzamentos (aplicação da penalidade)

Sucessive Shortest Path (DIJNOR)

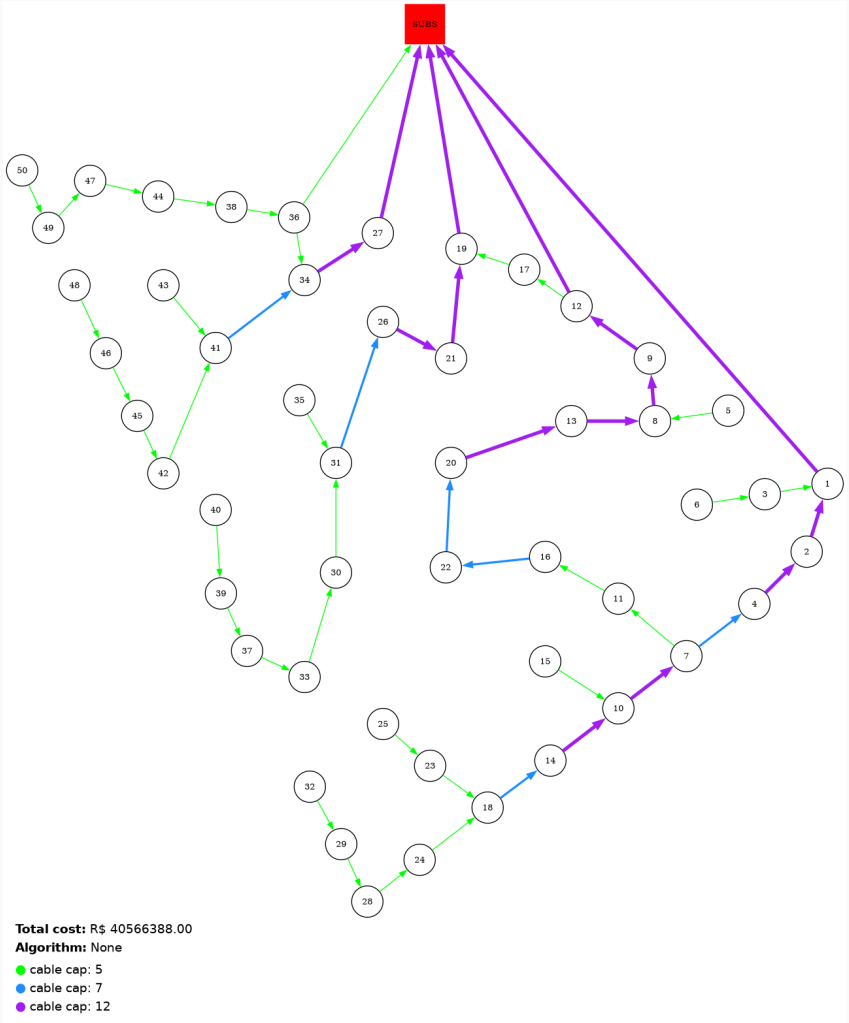
```
// Para cada turbina
for turbine in turbines {
    // Caminho
    let path = find_simple_forward_path(problem, turbine, *substation, &active_flows);

    // Adiciona 1 de fluxo ao longo do caminho
    for i in 0..(path.len() - 1) {
        let u = path[i];
        let v = path[i + 1];

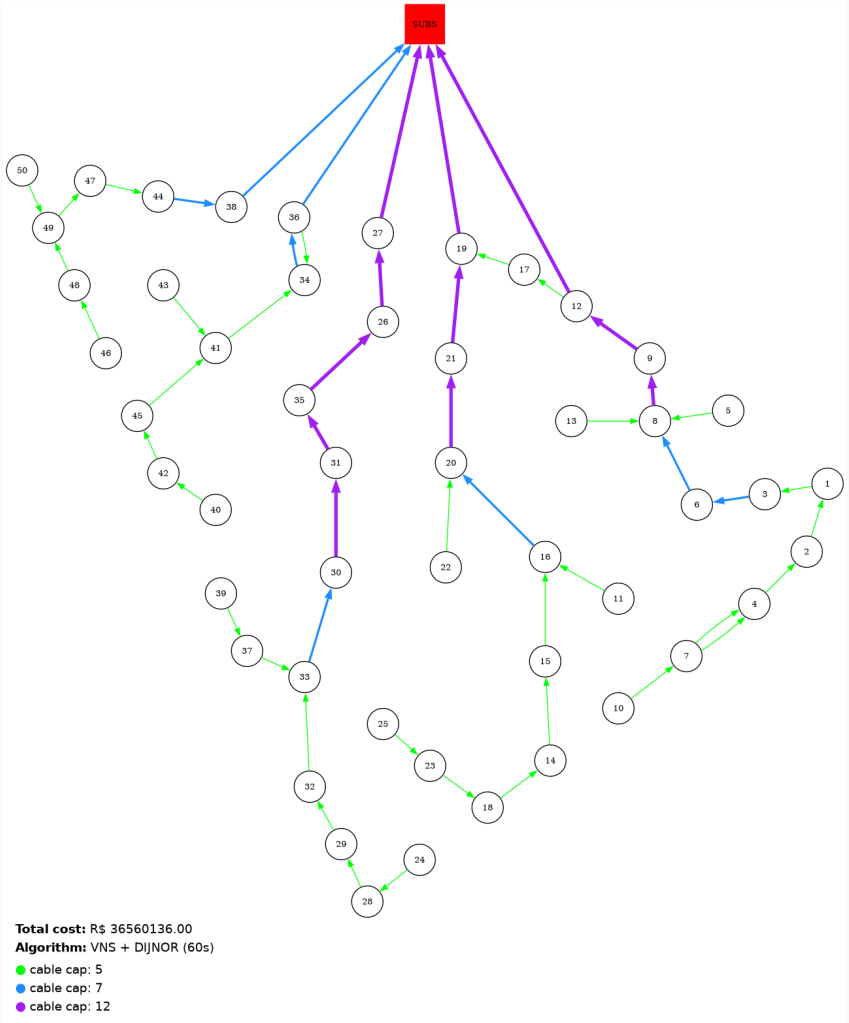
        // Pega o fluxo atual na direção contrária (v → u)
        let inverse_flow = *active_flows.get(&(v, u)).unwrap_or(&0);
        let direct_flow = *active_flows.get(&(u, v)).unwrap_or(&0);

        if inverse_flow > 0 {
            // Se já existe fluxo voltando, nós CANCELAMOS 1 unidade dele
            active_flows.insert((v, u), inverse_flow - 1);
        } else {
            // Caso contrário, apenas adicionamos na direção de ida
            active_flows.insert((u, v), direct_flow + 1);
        }
    }
}
```

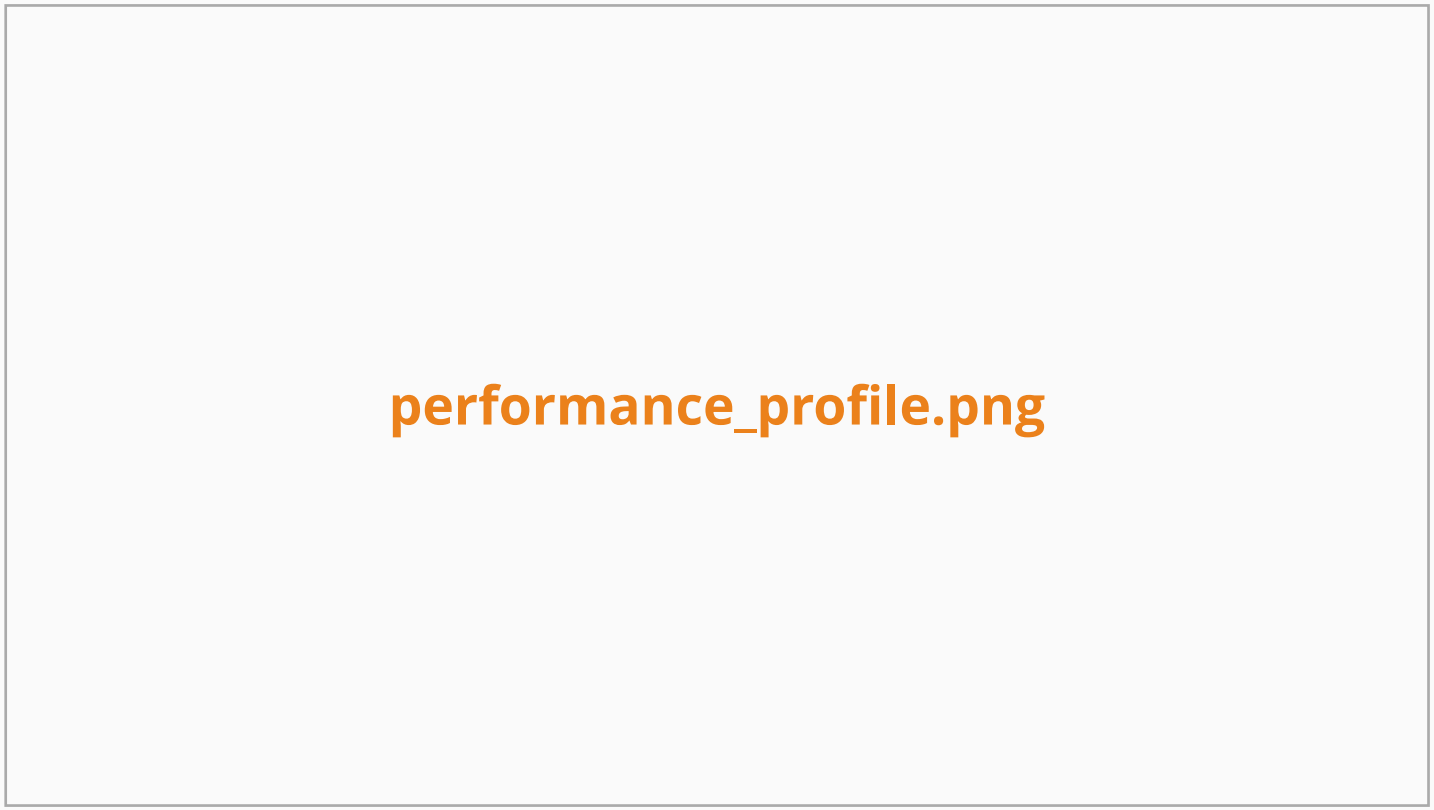
Sucessive Shortest Path (DIJNOR)



Sucessive Shortest Path (DIJNOR) + VNS



Comparativo geral



Comparativo geral

Algoritmo	Melhor Custo (Milhar)	Custo Médio (Milhar)	Tempo Médio (ms)	Execuções
VNS + Sweeping-MultiStart (60s)	28811.8	71720.4	60009.97	221
VNS + Dijnor (60s)	28994.13	83588.74	60019.99	220
SweepingMulti-start	29555.48	73251.25	15.33	221
Sweeping	31032.8	78543.95	0.0	222
Dijnor	34678.66	84687.19	116.38	232

Comparativo geral

Acredito que o SSP realmente seja inferior, mais caro e com resultados piores.

Comparativo geral

Acredito que o SSP realmente seja inferior, mais caro e com resultados piores.

Mas parecem existir espaços para melhorias, talvez aplicar processos de melhoria das soluções a partir do SSP seja um caminho.

Obrigado pela atenção!